

CAP2791C: POWER BI DATA PREPARATION AND MODELING

DAX Fundamentals: Measures, Functions, and Filter Context

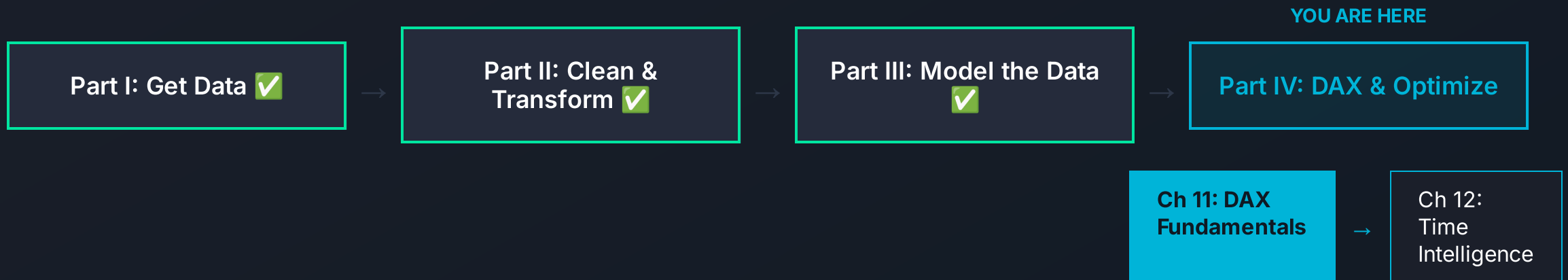
From Cleaned Data to Calculated Answers

Professor Carlos Marquez

```
Total Sales = SUM(Sales[Amount])
```

Where Are We in the Pipeline?

Part IV begins here



✓ **Parts I–III complete:** Your data is clean, combined, and modeled.

📐 **Part IV focus:** Write calculations that answer business questions.

- **This chapter:** 8 measures, filter context, and CALCULATE.

Your Data Can't Answer Questions Without Calculations

The difference between data and insight

- A table of transactions is raw material — it records events, but it doesn't answer questions.
- "What is the total revenue?" → needs **SUM**
- "What is the average transaction?" → needs **AVERAGE**
- "How does one segment compare to the whole?" → needs **CALCULATE**
- **DAX** is the language that turns raw records into answers.

Date	Truck	Amount
10/1	T001	\$12.50
10/1	T002	\$8.00
10/1	T001	\$15.00

↓ **DAX** ↓

TOTAL REVENUE
\$35.50

M vs. DAX: Two Languages, Two Jobs

The prep cook and the line cook

Feature	M (Power Query)	DAX
Where it runs	Power Query Editor	Main Power BI window (formula bar)
When it runs	During data refresh	When a visual renders
What it does	Shapes, cleans, combines data	Calculates aggregations, ratios, filtered results
Analogy	Prep cook (before service)	Line cook (during service)

CROSS-TOOL BRIDGE

From SQL: M $\approx$ ETL scripts; DAX $\approx$ aggregate functions with WHERE clauses

From Excel: M $\approx$ data cleanup before a PivotTable; DAX $\approx$ calculated fields inside the PivotTable

The DAX Measure Pattern: English First, DAX Second

Every measure starts with a plain-English question

1 State the question in plain English

"What is the total of all sales amounts?"

2 Identify the function

SUM, AVERAGE, COUNT, MIN, MAX,
DIVIDE, DISTINCTCOUNT

3 Write the DAX

```
Total Sales = SUM(Sales[Amount])
```

Function	Plain English	Returns
SUM	Add up all values in a column	A total
AVERAGE	Find the average value	A mean
MIN / MAX	Find the smallest / largest value	A single value
COUNTROWS	Count how many rows are in a table	A count
DISTINCTCOUNT	Count unique values only	Unique count
DIVIDE	Divide safely (no error on zero)	A ratio

Explicit Measures vs. Implicit Aggregation

Always write the measure — never let Power BI guess

Implicit (Avoid)

- ✗ Drag a numeric column directly into a visual
- ✗ Power BI auto-aggregates (usually SUM)
- ✗ No name, no format, no documentation
- ✗ Can change unexpectedly
- ✗ Not reusable across visuals

Explicit (Preferred)

- ✓ Write a named DAX formula:

```
Total Sales = SUM(Sales[Amount])
```

- ✓ You control the aggregation, name, and format
- ✓ Consistent across every visual in the report
- ✓ Reusable — write once, use everywhere

PROFESSIONAL STANDARD

In professional Power BI development, teams rely on explicit measures exclusively. Implicit aggregations are a source of inconsistency and reporting errors.

Writing Your First Measure: SUM and Card Visual

The core workflow for every measure in this chapter

Data pane → [Table name] → Home tab → New Measure → formula bar → type formula → Enter → Visualizations pane → Card → drag measure in

- 1 Select the table in the Data pane first (this sets the "owner" of the measure).
- 2 Type the formula in the formula bar: `Total Sales = SUM(Sales[Amount])`
- 3 The measure appears with a **calculator icon** (⚙️) — not a sigma (Σ) like a column.
- 4 Drag the measure into a Card visual → a number appears.

✓ VERIFICATION CHECK

The Card shows a large number. If you see the measure name but no number, you dragged the column (Σ) instead of the measure (⚙️). If you see an error, check that the table name and column name are spelled exactly as they appear in the Data pane.

COUNTROWS vs. DISTINCTCOUNT: Two Ways to Count

Counting rows is not the same as counting unique values

COUNTROWS(Sales)

- Counts every row in the table
- One row = one transaction
- **Returns:** total number of transactions

Example: 4,200 transactions

DISTINCTCOUNT(Sales[Menu Item ID])

- Counts unique values in one column
- Ignores duplicates
- **Returns:** how many different items were sold

Example: 12 unique menu items

ANALOGY

Think of counting plates served at a buffet (**COUNTROWS**) vs. counting how many different dishes were offered (**DISTINCTCOUNT**). Same event, completely different questions.

COMMON MISTAKE

COUNT vs. COUNTROWS: COUNT skips blank values in a column. COUNTROWS counts every row regardless of blanks. When counting transactions, always use COUNTROWS.

DIVIDE: Safe Division That Never Crashes Your Report

Handling the "divide by zero" problem gracefully

Formula Anatomy

```
Tip Percentage =  
    DIVIDE(  
        SUM(Sales[Tip]), ← divide safely  
        SUM(Sales[Amount])  
    )  
    numerator: total tips  
    denominator: total sales
```

Why DIVIDE instead of /

- Regular division (/) throws an error when the denominator is zero.
- DIVIDE** returns a **blank** instead of crashing the visual.
- Optional third argument:**

```
DIVIDE(num, den, alternate_result)
```

Specify what to show instead of blank.

CROSS-TOOL BRIDGE

Excel Equivalent

```
=IFERROR(A1/B1, 0)
```

SQL Equivalent

```
CASE WHEN SUM(Amount) = 0 THEN NULL  
ELSE SUM(Tip)/SUM(Amount) END
```

DAX

DIVIDE does this in one clean function

Filter Context: The Invisible Force Behind Every Measure

Why one measure produces different numbers in different visuals

The Room-and-Doors Metaphor

- All your data is in a large room.
- Every slicer, row header, column header, and filter pane entry is a door.
- When a filter is active, some doors close — the measure only sees what's left in the room.
- Your measure doesn't know or care what's behind the closed doors — it calculates whatever is visible.

1

Card visual, no filters

All doors open → **Grand total**

2

Bar chart with category on axis

One "room" per category → **Different total per bar**

3

Slicer selected

Doors close for non-selected values → **Filtered total**

KEY INSIGHT

You write **ONE** measure. The visual creates the filter context. The measure responds automatically. This is why a single **Total Sales** measure works correctly in a Card, a bar chart, a table, and a filtered slicer — all at the same time.

Row Context vs. Filter Context: Know the Difference

Measures use filter context. Calculated columns use row context.

Filter Context (Measures)

- Applies when Power BI renders a visual
- Determines which rows are "in the room"
- Set by slicers, row headers, column headers, filters

***Example:** Total Sales in a bar chart — each bar has its own filter context.*

Row Context (Calculated Columns)

- Applies when Power BI evaluates a calculated column
- Power BI walks through the table row by row
- "For THIS row, what is the value?"

***Example:** A Revenue Category column that checks each row's Amount and assigns "High/Medium/Low".*

THE GOLDEN RULE

In this chapter, we focus on **filter context** because we are writing measures. Row context matters for calculated columns (which you built in Chapter 10). The distinction becomes critical when you start using CALCULATE.

CALCULATE: Override the Filter Context

The most important DAX function you will learn

FORMULA ANATOMY

Filtered Measure =

```
CALCULATE(  
    SUM(Sales[Amount]),  
    Sales[Category] = "Entrée"  
)
```

← *take this calculation...*
← *(the sum of all sales amounts)*
← *...and apply THIS filter instead*

What CALCULATE does:

- 1 Takes a calculation (any DAX expression)
- 2 Applies a filter override — ignores what the visual is doing and forces a specific filter
- 3 Returns the result of the calculation under the new filter

✓ VERIFICATION CHECK

Put a CALCULATE measure and a regular SUM measure side-by-side in a Table visual. The regular SUM changes per row

🔗 SQL BRIDGE

CALCULATE is like adding a WHERE clause to a SUM: `SELECT SUM(Amount) FROM Sales WHERE Category = 'Entrée'.`

View Compass: Where You Work in This Chapter

Report View is your primary workspace

View	What You Do Here	This Chapter?
Report View	Write measures, build visuals, verify results	✓ Primary
Table View	Inspect data values, check calculated columns	✓ Reference
DAX Query View	Test and debug DAX expressions	✓ Introduced
Model View	Relationships (already built)	Not needed
Power Query Editor	Clean and transform (already done)	Not needed

WORKSPACE TIP

The **formula bar** (white text area below the ribbon) is where you type all DAX expressions. If you don't see it: go to the **View tab** → check the box next to **Formula Bar**.

Common DAX Errors and How to Fix Them

Errors are clues, not failures

"A single value cannot be determined"

CAUSE

You referenced a column without an aggregation function. You wrote: `Sales[Amount]`

FIX

Wrap the column in a function: `SUM(Sales[Amount])`

"Column already exists in the table"

CAUSE

Your measure name matches an existing column name in the table.

FIX

Use a descriptive phrase (e.g., "Total Sales"), not a column name ("Amount").

Measure shows blank instead of a number

CAUSE

The current filter context eliminated all data (there are no rows left in the "room").

FIX

Clear all slicers and filters — if a number appears, a filter was hiding the data.

System Check: Your Measure Library Is Built

Ready for Time Intelligence in Chapter 12

- ✓ **M vs. DAX understood** — M cleans data before the model; DAX calculates inside the model
- ✓ **8 measures written** — SUM, AVERAGE, MIN, MAX, COUNTROWS, DISTINCTCOUNT, DIVIDE, CALCULATE
- ✓ **Explicit measures only** — named, formatted, reusable
- ✓ **Filter context grasped** — visuals set the context; measures respond automatically
- ✓ **CALCULATE mastered** — override filter context for specific subsets
- ✓ **Errors diagnosed** — you know the 3 most common errors and their fixes